

Rider University
Norm Brodsky College of Business

CIS 360

Final Project

Names: Vanessa Vega & Nishtha Patel

Instructor:
Dr. Mashayekhi

Format: Times New Roman (font size =12), single-space.

Please export the process in RM and submit them with this file.

Part 1: Business Understanding (Problem): (5 marks)

Explain in a few sentences what we try to predict and how it helps a hotel manager manage bookings.

The goal of this model is to predict whether a hotel booking will be cancelled before it happens. We are using the `is_cancelled` label where 1 indicates a cancellation, and 0 indicates the guest will follow through with their stay. By analyzing patterns in past booking data, the model attempts to flag which current reservations are at risk of being cancelled before it's too late to act on them.

This predictive capability is valuable to hotel managers because cancellations are one of the biggest challenges in hospitality management. Having a model that can predict when someone is likely to cancel their reservation gives managers the ability to be proactive. They can make smarter overbooking decisions since they would have a better idea of how many bookings will cancel. They also having this information allows them not to miss out on revenue. Overall, this helps keep occupancy rates steady and revenue flow much more predictable.

Part 2: Data Understanding (5 marks)

2-1 what is the sample size?

a. 119,390

2-2 Complete the following tables:

Variable	definition of the variable*	Average	Standard Deviation	Min	Max
adr	Average daily rate	101.831	50.536	-6.380	5400
Lead_time	Number of days that elapsed between the entering date of the booking into the PMS and the arrival date	104.011	106.863	0	737
adults	Number of Adults	1.856	.579	0	55
children	Number of Children	.104	.399	0	10
days_in_waiting_list	Number of days the booking was in the waiting list before it was confirmed by customer	2.321	17.595	0	391

See Table-1 in the instruction file

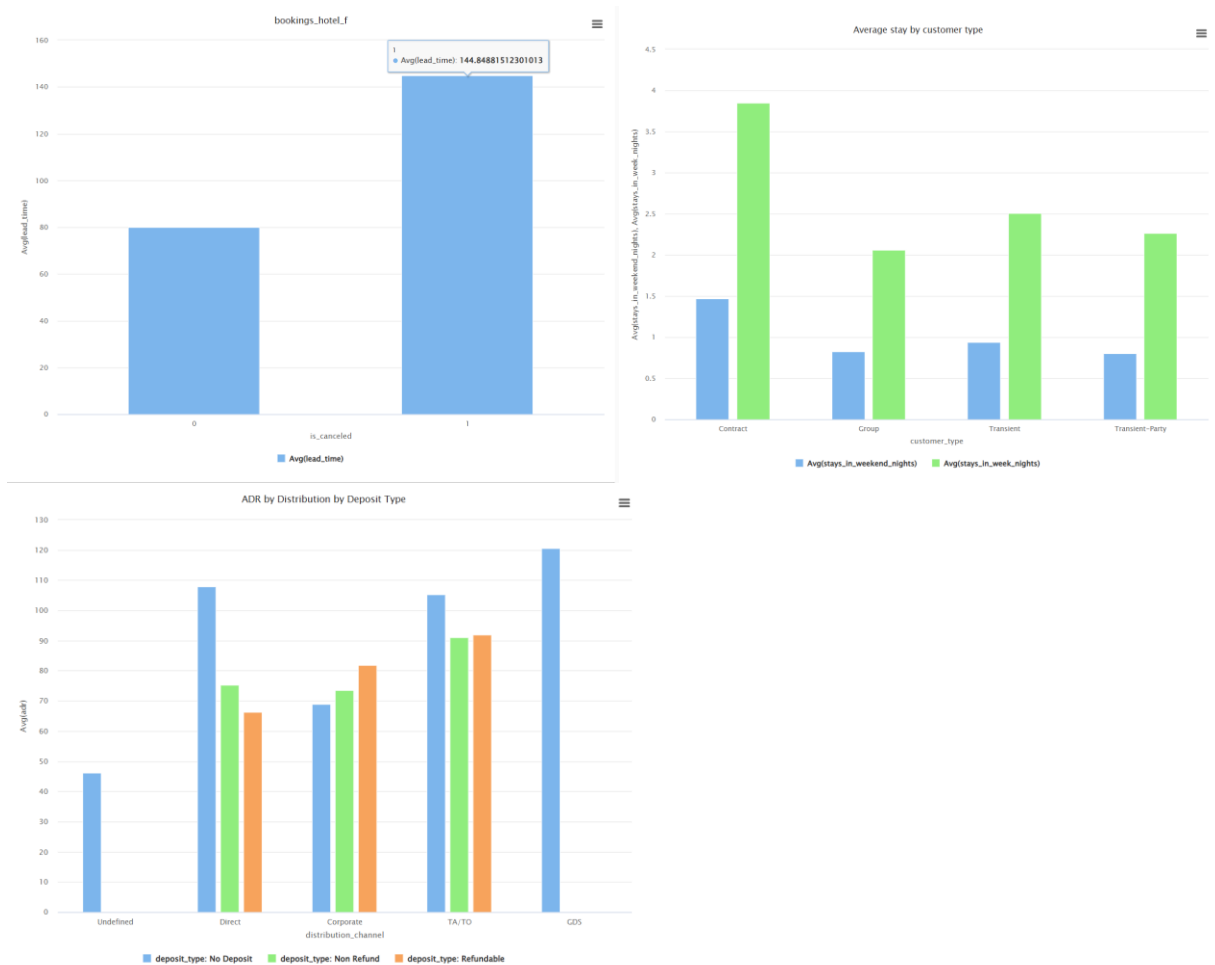
Deposit_type	Absolute Frequency (count)	Relative Frequency (fraction) %	Cumulative Percentage %
No Deposit	104641	.876	87.6%
Non Refund	14587	.122	99.8%
Refundable	162	.001	100%

meal	Absolute Frequency (count)	Relative Frequency (fraction) %	Cumulative Percentage %
BB	92310	.773	77.3%
HB	14463	.121	89.4%
SC	10650	.089	98.3%
Undefined	1169	.010	99.3%
FB	798	.007	100%

Is_canceled	Absolute Frequency (count)	Relative Frequency (fraction) %	Cumulative Percentage %
0	75166	.630	63%
1	44224	.370	100%

2-3 Include at least 3 graphs: (using Tableau or RM)

Examples for graphs: A bar chart showing is_canceled vs. lead_time (average)



Part 3: Data Preparation (10 Marks)- Nishtha

Conduct all preparation tasks needed to make the data ready for analysis, including but not limited to handling outliers, missing values, and inconsistent data, creating new variables, and removing variables from the model. For each data preparation task, explain why you need to do that clearly and briefly.

Data Preparation Tasks

Data Preparation Task	Variable(s)	Justification
Replace Missing Values – children	children	Action taken: Replaced missing values with the column average The children column had 4 missing values. We replaced the missing values with the average preserves the distribution without removing valid booking records.
Filter Missing Values – country	country	Action taken: Removed 488 rows where county was missing Country of origin is an important predictor of cancellation behavior. Since only 0.4% of records had missing country values, removing them has minimal impact on the data.
Filter Undefined Categories	meal, market_segment, distribution_channel	Action taken: Removed rows where any of these columns contained 'Undefined' “Undefined” is not a meaningful category and provides no predictive value. The records represent incomplete or erroneous data

		entries that could mislead the model.
Filter Negative ADR Values	adr	Action Taken: Removed rows where $ADR < 0$ A negative average daily rate is logically impossible for a hotel booking. These values are almost certainly data entry errors that would distort the model's understanding of pricing.
Filter ADR Outliers	adr	Action Taken: Removed rows where $ADR > 400$ Standard hotel rates typically range from \$50–\$200 per night. Rates above \$400 are extreme outliers likely representing luxury suites or data errors. Removing them improved model for typical bookings.
Filter Zero Adults	adults	Action Taken: Removed rows where $adults = 0$ A hotel booking with zero adults is logically impossible and represents ghost or error records. These 403 records were removed to maintain data integrity.

Filter Adult Outliers	adults	Action Taken: Removed rows where adults > 10 A maximum of 55 adults in the original data suggests data errors. Values above 10 are extremely rare and likely erroneous, so they were removed to reduce noise.
Create New Variable – adults_cat	adults	Action Taken: Created adults_cat: small (1–2), medium (3–5) Categorizing the number of adults converts a continuous variable into meaningful groups that better capture booking behavior patterns. 'Small' bookings (1–2 adults) represent 94.7% of all bookings, making this distinction useful for the model.
Remove babies Column	babies	Action Taken: Dropped the babies' column entirely Over 99% of values in this column are 0, making it nearly useless as a predictor. Including it would add noise to the model without providing meaningful information.
Filter Children Outliers	children	Action Taken: Removed rows where children > 3 Used a visual and it showed that having more than 3 children in a hotel booking is extremely rare and likely a data entry error.

Create New Variable – Top_10_Country	country	Action Taken: Created top 10 countries The original <i>country</i> variable contained 178 unique values, which made it too high-cardinality for an effective decision tree model. To address this, we grouped the data into the top 10 most frequent countries, with all remaining countries combined into an “Other” category. This reduced complexity while still preserving meaningful geographic patterns that could help predict cancellation behavior.
Remove country Column	country	Action Taken: Dropped original country column With 178 unique values, the country column would create overly complex Decision Tree splits. The new grouped top_10_country variable captures geographic patterns more cleanly and improves model interpretability.
Remove days_in_waiting_list Column	days_in_waiting_list	Action Taken: Dropped the days_in_waiting_list column 97% of values in this column are 0, indicating that the vast majority of bookings had no waiting list period. This near-zero-variance variable provides minimal predictive power and was removed to reduce noise.

Filter lead_time Outliers	lead_time	Action Taken: Removed rows where lead_time > 400 Booking more than 400 days in advance is extremely uncommon for hotel reservations. These records likely represent system errors or highly unusual cases that would not generalize well in a predictive model.
Filter stays_in_week_nights Outliers	stays_in_week_nights	Action Taken: Removed rows where stays_in_week_nights > 14 Stays exceeding 14 weeknights (2 weeks) are highly unusual for hotel bookings and likely represent long-term stays or data errors. Removing these outliers improves model accuracy for typical booking patterns.
Filter stays_in_weekend_nights Outliers	stays_in_weekend_nights	Action Taken: Removed rows where stays_in_weekend_nights > 6 More than 6 weekend nights in a single booking is atypical and suggests data errors. Filtering these outliers ensures the model learns from realistic booking patterns.

Filter required_car_parking_spaces Outliers	required_car_parking_spaces	Action Taken: Removed rows where required_car_parking_spaces > 3 When we looked at the data, we created a visual of a barchart to showcase the outliers. Anything after 3 was rare and was not realistic for standard hotel bookings.
Set Role – is_canceled	is_canceled	Action Taken: Set is_canceled as the label (target variable), positive class = 1 is_canceled is the outcome we are trying to predict. Setting it as the label tells RapidMiner to use it as the dependent variable in the predictive model.
Numerical to Binominal – is_canceled, is_repeated_guest	is_canceled, is_repeated_guest	Action Taken: Converted both variables from numeric to binominal Both variables are binary (0/1) but were imported as numeric. Converting them to binominal ensures RapidMiner treats them as categorical variables, which is required for correct classification modeling.

Summary of Changes

Original dataset: 119,390 rows | 23 columns

Final cleaned dataset: 115,449 rows | 23 columns (2 new variables created, 3 removed)

Part 4: Modeling (10 marks)

4-1 What is the target variable (label)?

- a. The target variable is “is_canceled”.

4-2 What is the positive class of the target variable?

The positive class is 1.

4-3 Why do we need to use classification instead of regression?

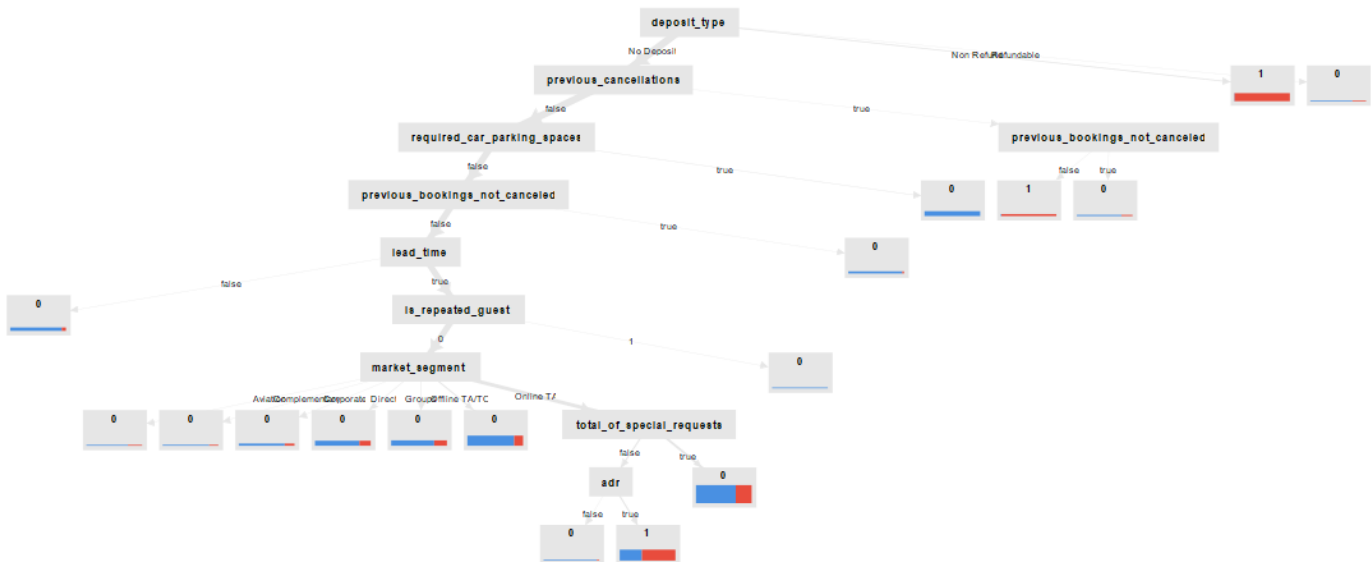
Regression and classification are both supervised learning methods, but they are used for different types of problems. Regression is used when the goal is to predict a continuous value. In this specific dataset of hotels this could be estimating how much revenue a booking will generate or what price a room should be set at. These kinds of outcomes are measured on a numerical scale. Classification, on the other hand, is used when the outcome falls into specific groups or categories. Instead of predicting a number, it assigns an observation to a label such as yes or no, or cancelled versus not cancelled. In this hotel dataset, the target variable (is_cancelled) only has two possible values. A booking either gets cancelled or it does not, so treating it like a continuous value would not make sense. Because of this, classification is the better choice. Hotel managers are not trying to predict a numerical value, they need a clear indication of whether a booking is likely to be cancelled so they can make decisions ahead of time, such as adjusting pricing strategies or planning for possible overbooking. Overall, regression is best when you are working with measurable quantities, but classification fits better here because both the outcome and the decisions based on it are categorical.

4-4 Describe the decision tree for the model (how many nodes, how many terminal nodes, depth, variables in the tree)

The total number of nodes (including the root node)?	27
The number of terminal nodes?	17
The depth (do not include the root node)?	8
List the name of variables in the tree	Deposit_type Pervious_cancellation Required_car_parking_spaces Pervious_booking_not_cancelled Lead_time Is_repeated_guests Market_segment Total_of_special_request

	adr Pervior_booking_not_cancelled
What is the most important predictor of is_canceled: Yes, based on the tree?	Deposit_type

4-5 Copy and paste the decision tree here (take a screenshot, resize it and paste it here. Make sure the image is readable)



Part 5: Model Evaluation (10 marks)

5-1 Complete the confusion matrix based on the “Performance Test” results:

Predictive Model for Hotel Cancellation		Actual	
		Yes (1)	No (0)
Predicted	Yes	TP= 7808	FP= 2228
	No	FN= 4844	TN= 19755

5-2 Complete the following table: (positive class: Yes)

	General Accuracy	Recall	Precision
Performance - Cross Validation	79.11%+/-1.51%	55.99%+/-11.27%	82.92%+/-9.21%
Performance - Test	79.58%	61.71%	77.80%

Note: for performance-CV, you should report average and standard deviation (e.g., recall: 79.41% +/- 4.39%)

5-3 Describe two groups of is_canceled: Yes and two groups of is_canceled: No based on the features in this model

	Describe
Group 1 (Yes)	Non Refund Deposit Bookings Any booking with a non-refundable deposit is predicted as cancelled because the model identified a strong pattern in the data where these bookings still frequently ended up being cancelled.
Group 2 (Yes)	No Deposit, Previous Cancellations, No Completed Booking Each condition highlights risky booking behavior. No Deposit means there is no financial commitment, Previous Cancellations = true shows a history of cancelling, and Previous Bookings Not Cancelled = false indicates no record of completed stays. Together, this pattern reflects unreliable behavior with no financial penalty, so the model classifies this group as likely to cancel.
Group 1 (No)	No deposit, No previous cancellations, No parking, No previous bookings not cancelled, Short lead time (All variables = false). This combination suggests a more reliable and committed booking pattern. The absence of any cancellation

	history and consistent past behavior indicates dependability, while the short lead time suggests the booking was made with more certainty. Based on this, the model classifies these guests as low risk and unlikely to cancel.
Group 2 (No)	No deposit, No previous cancellations(=false), Parking requested(=true) Even though there is no deposit, the fact that these guests have no history of cancellations suggests consistent and reliable behavior. In addition, requesting parking indicates a higher level of planning and commitment to the stay, since they are thinking about logistics in advance. When these factors are combined, the model interprets this group as lower risk and predicts that they are less likely to cancel.

Part 6: Answer the following questions. (15 marks)

6-1 Is this model underfitting, overfitting, or just the right fit? Why?

The model shows signs of slightly overfitting because the decision tree is a complex, with multiple levels and splits that allows it to capture very specific patterns in the training data. This can help improve the performance on the training set and it can reduce how well the model generalizes to new data. When comparing the cross-validation results to the test results, the performance is relatively close, which suggests that the model is not severely overfitting. However, after the optimization, the model becomes deeper and more detailed, and although recall increases significantly, accuracy and precision decrease. This trade-off indicates that the model is starting to focus too much on fitting the training data, especially to detect cancellations, rather than maintaining balance overall performance.

6-2 Provide at least two suggestions to improve the model. (Please note that the predictive model is supposed to detect is_canceled: Yes)

One important step would be to control how complex the decision tree becomes. With the current decision tree, it grows deeper and helps capture detailed patterns but also increases the risk of overfitting. When a model becomes too complex, it starts learning noise instead of real patterns, which reduces its ability to perform well on new data. This can be improved by slightly limiting the maximum depth of the trees and increasing the minimum leaf size and minimum split size. Doing this would force the model to focus on more meaningful and general patterns in booking behavior rather than very specific cases.

Another way to improve that model would be to perform feature selections by removing less important variables from the decision tree. Based on the tree structure, some variables may not contribute significantly to predicting cancellations but still add complexity to the model. Keeping unnecessary variables can make the model more complicated without improving performance. By focusing only on the most important, such as `deposit_type` and `previous_cancellations`, the model can become more efficient and easier to interpret. This can also help reduce overfitting and improve how well the model generalizes to new booking data.

6-3 Explain which error (FP or FN), in your opinion, should be prioritized to minimize. Explain the consequences of your choice (give priority to FP or FN) regarding overbookings and reducing cancellations.

FN should be prioritized to be minimized in this scenario. In the context of hotel management, a false negative occurs when the model predicts that a guest will NOT cancel, but they actually do. This is a costly mistake in a hotel management setting. When the model fails to flag a real cancellation the hotel misses out on taking action because they assumed it would not be cancelled. The room sits empty and there is no replacement guest lined up, therefore leaving them missing out on revenue. This is crucial with hotels as rooms being occupied is the main way they get their money and an empty room represents pure loss with no way to recover it.

The specific consequence tied directly into the overbooking strategy is that hotels have an industry practice to overbook rooms to account for expected cancellations, but this strategy only works if the model is correctly identifying which bookings are likely to cancel. If false negatives are high, the hotel is underestimating how many cancellations will actually happen, which means they are not overbooking enough to compensate. The result of this leads to consistent revenue loss and more empty rooms.

On the other hand, a false positive, predicts a cancellation that doesn't actually happen, this is not as serious as a mistake. Most of the time this situation is recoverable whether its finding nearby accommodations or offering upgrades to maintain their guest relationships. The false positive is more of an inconvenience than a revenue loss

This is why in our part 8 process the MetaCost operator was used with a cost matrix assigning 2.3 to FN vs 1 to FP, this setup encodes the idea that a missing cancellation is more than twice as costly as a false alarm.

Part 7: Based on the confusion matrix for test data (in part 5) and the following information, complete the following table: (5 mark) – All numbers should be rounded to the nearest integer.

Number of bookings=10000

Net revenue for each room = \$150

7.1 How many bookings will be predicted as Yes (canceled)?	$10,000 \times (10,036/34,635) = 2,898$
7.2 How many bookings will be predicted as No (not canceled)?	$10,000 \times (24,599/34,635) = 7,102$
7.3 Of those predicted as Yes, how many of them are FP?	$2,898 \times (2,228/10,036) = 643$
7.4 Of those predicted as No, how many of them are FN?	$7,102 \times (4,844/24,599) = 1,399$
7.5 How many rooms will likely be overbooked?	Overbooked = FP in new bookings = 643
7.6 How many rooms will likely remain empty?	Empty = FN – Overbooked = $1,399 - 643 = 756$
7.7 How much revenue will the hotel lose, assuming that the booking system automatically assigns the overbooking customers to available rooms?	$756 \times \$150 = \$113,400$

Part 8: (50 Marks)

To improve the model, use optimization. Describe the results clearly, including

- (1) Explain how to set parameters for optimization – Provide the list of hyperparameters and their values you assign.

In terms of the physical setup in RapidMiner, the Optimize Parameters (Grid) operator was placed as the outermost operator, wrapping the entire Cross Validation. Inside the Cross Validation, the structure was MetaCost, Apply Model, Performance (Binominal Classification), with the Decision Tree nested inside MetaCost itself. This is a key structural detail, by placing the Decision Tree inside MetaCost rather than directly in the Cross Validation, the cost matrix of 2.3 for false negatives directly influences how the tree learns during training, not just how predictions are evaluated afterward.

To select which parameters to tune, I opened Edit Parameter Settings within the Optimize Parameters operator. This brought up a three-panel window where I selected Decision Tree (4) from the Operators list, which then displayed all its available parameters in the middle panel. The four hyperparameters were then moved across into the Selected Parameters panel on the right using the arrow buttons, and each one was configured individually in the Grid/Range section at the bottom. The criterion was handled through the Value List since it is categorical, while the three numeric parameters each had their Min, Max, Steps, and Scale values entered directly. Linear scale was used for all three, producing evenly spaced values across each range

To improve the model's ability to correctly identify hotel cancellations, we decided to focus on increasing recall. We chose this because we wanted to focus on false negatives. Recall focuses on catching these, so we used the optimized parameters grid operator. The purpose of the grid search is to test many different combinations of hyperparameters instead of relying on intuition alone.

Within the edit parameter settings section of the optimize parameters operator, the decision tree model was selected. We chose four key hyperparameters because each one directly influences how the decision tree learns patterns in data, which I will explain below.

The first thing we chose was the criterion which is a split quality measure. This controls how the algorithm decides the best way to split the nodes in the tree. All available options were tested besides least square. So, the tested options we chose were `gain_ratio`, `information_gain`, `gini_index`, and `accuracy`. This was to determine which split method best handles the structure of the hotel cancellation dataset. Each criterion behaves differently, especially dealing with categorical variables and class imbalance, so testing all options was necessary to avoid bias toward a single assumption.

The second parameter we added was minimal leaf size we did minimum 2 to maximum 20 and step size 5. This parameter controls how many observations must exit in a terminal node. Smaller values allow the model to create more detailed branches, which capture subtle cancellation patterns, while larger values force the model to generalize more. The range (2,6,9,13,16,20) was chosen to balance flexibility with overfitting control.

The third parameter we added was maximal depth we did minimum 5 to maximum 20 and the step size remained as 5. This parameter determines how deep the decision tree is allowed to grow.

Deeper trees can capture more complex relationships in the data but may also overfit. Testing values from 5 to 20 allowed the model to explore simplified and highly detailed structures.

The fourth parameter was minimal size for split minimum was 5 the maximum was 20 and the step size again was 5. This parameter controls the minimum number of examples required before a node can be split. Lower values allow more splits and more detailed segmentation of the dataset, while higher values reduce unnecessary branching.

Overall, the grid evaluated 864 different hyperparameter combinations (4 criteria x 6 leaf sizes x 6 depths x 6 split sizes). The optimization goal was set to maximize recall and a MetaCost matrix was applied to reflect business priorities, assigning a higher penalty of 2.3 to false negatives than false positives we put it at 1.0. The results were sorted highest to lowest recall to identify the best combination. This ensured that the model prioritized catching actual cancellations, even at the cost of more false alarms.

Insert the updated table 5-1 and 5-2

The confusion matrix based on the “Performance Test” results:

Predictive Model for Hotel Cancellation		Actual	
		Yes (1)	No (0)
Predicted	Yes	TP= 11000	FP= 8585
	No	FN= 1652	TN= 13398

Complete the following table: (positive class: Yes)

	General Accuracy	Recall	Precision
Performance - Cross Validation	70.99%+/-0.47%	87.39%+/-1.15%	56.68%+/-0.44%
Performance - Test	70.44%	86.94%	56.17%

Note: for performance-CV, you should report average and standard deviation (e.g., recall: 79.41% +/- 4.39%)

(2) Describe the optimum hyperparameters for the model (insert a table that shows the optimum values for hyperparameters)

The best-performing configuration was identified at iteration 121. The highest maximum recall of 0.874 during the grid search. The hyperparameters for this configuration were criterion that was set to gain_ratio, minimal leaf size was 2, maximal depth was 20, and minimal size for split is 5. The results in the table below show a clear pattern among the best-performing models that the gain ratio criterion consistently outperformed the other splitting methods.

This is because gain ratio adjusts for the bias toward attributes with many categories, which is especially important in this dataset where variables like booking source, customer type, or country have high cardinality. Without this adjustment, the model may place too much importance on these

features simply due to the number of categories they contain, which can lead to poorer generalization on new data.

The minimal leaf size of 2 allows very small terminal nodes. This allowed the model to capture more detailed patterns in cancellation behavior, suggesting that cancellations are driven by small, specific combinations of features that might be overlooked in a more generalized tree. Similarly, a maximum depth of 20 allowed the tree to grow sufficiently complex to model nonlinear relationships in the data. This depth helped capture interactions between variables that simpler models would miss. Finally, setting the minimal split size to 5 ensured that each split was based on a sufficient number of observations, helping prevent the tree from becoming too fragmented.

When combined with the MetaCost adjustment, this configuration produced a model that successfully identifies approximately 87% of all actual cancellations, a substantial improvement over the baseline recall of 55.99%. This improvement showcases the hyperparameter tuning conducted, which improved predictive performance.

Optimize Parameters (Grid) (2) (864 rows, 6 columns)

iteration	Decisio...	Decisio...	Decisio...	Decisio...	recall ↓
121	gain_ratio	2	20	5	0.874
265	gain_ratio	2	20	8	0.872
241	gain_ratio	2	17	8	0.855
97	gain_ratio	2	17	5	0.831
409	gain_ratio	2	20	11	0.830
529	gain_ratio	2	17	14	0.811
697	gain_ratio	2	20	17	0.809
841	gain_ratio	2	20	20	0.809
385	gain_ratio	2	17	11	0.791
553	gain_ratio	2	20	14	0.789
673	gain_ratio	2	17	17	0.749
817	gain_ratio	2	17	20	0.727
217	gain_ratio	2	14	8	0.725
361	gain_ratio	2	14	11	0.725
269	gain_ratio	6	20	8	0.694
821	gain_ratio	6	17	20	0.694
557	gain_ratio	6	20	14	0.694
413	gain_ratio	6	20	11	0.692
649	gain_ratio	2	14	17	0.687
70	gain_ratio	2	14	5	0.684

You should attach the RM process for part 8 separately (name the file as part 8 process).